■ 2.1.13 Advanced Topic: Shorter Forms for Pure Functions

By using pure functions, you can avoid giving explicit names to functions. You can also avoid giving names to the variables or "formal parameters" that appear in these functions.

It is clear that the actual name `x` of the variable in the pure function `Function[x, x^2]` is entirely irrelevant. You can get a shorter form for the pure function by using the special object `#` as the variable, instead of the named symbol `x`. When `#` appears in a pure function, *Mathematica* assumes that it is the variable, without you explicitly saying so. As a result, `Function[#^2]` is equivalent to `Function[x, x^2]`.

The special object `#` is assumed to be the variable in the pure function `Function[#^2]`.

```
In[1]:= Function[#^2] [1 + a]
               2
Out[1]= (1 + a)
```

You can save even more typing by using a `&` instead of spelling out `Function`. The form *operation*`&` is equivalent to `Function[`*operation*`]`.

`#^2&` is equivalent to `Function[`*x*`, `*x*`^2]`.

```
In[2]:= #^2& [1 + a]
               2
Out[2]= (1 + a)
```

Using `#` and `&` allows for some very compact notation.

```
In[3]:= Map[#^2&, {a, b, c}]
          2   2   2
Out[3]= {a , b , c }
```

`#`	the first variable in a pure function
`#`*n*	the n^{th} variable
`##`	the sequence of all variables in a pure function
`##`*n*	the sequence of variables, starting with the n^{th} one
operation`&`	short form for `Function[`*operation*`]`; never forget the `&` when using this form

Short forms for pure functions.

Here is a pure function with two arguments.

```
In[4]:= (#1^2 + #2^2)& [x, y]
         2    2
Out[4]= x  + y
```

`##` stands for *all* the arguments you give.

```
In[5]:= f[##, ##]& [x, y]
Out[5]= f[x, y, x, y]
```